RESTAURANT ORDER & INVENTORY MANAGEMENT SYSTEM

SHAIK NASEEBA

104R BATCH

10000 CODERS

SQL QUERIES

DATABASE AND TABLE CREATION

CUSTOMERS TABLE

```
CREATE DATABASE Restaurant_Order_Inventory;
USE Restaurant_Order_Inventory;

CREATE TABLE Customers (
   customer_id INT PRIMARY KEY,
   name VARCHAR(100),
   phone VARCHAR(15),
   email VARCHAR(100),
   address VARCHAR(255)
);
```

MENU_ITEMS TABLE

```
CREATE TABLE Menu_Items (
   item_id INT PRIMARY KEY,
   item_name VARCHAR(100),
   category VARCHAR(50),
   price DECIMAL(10,2),
   availability_status VARCHAR(20)
);
```

ORDERS TABLE

```
CREATE TABLE Orders (
   order_id INT PRIMARY KEY,
   customer_id INT,
   order_date DATE,
   total_amount DECIMAL(10,2),
   status VARCHAR(20),
   FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);
```

ORDER_ITEMS TABLE

```
CREATE TABLE Order_Items (
   order_item_id INT PRIMARY KEY,
   order_id INT,
   item_id INT,
   quantity INT,
   subtotal DECIMAL(10,2),
   FOREIGN KEY (order_id) REFERENCES Orders(order_id),
   FOREIGN KEY (item_id) REFERENCES Menu_Items(item_id)
);
```

INVENTORY TABLE

```
CREATE TABLE Inventory (
   item_code INT PRIMARY KEY,
   ingredient_name VARCHAR(100),
   quantity_in_stock INT,
   unit VARCHAR(20),
   reorder_level INT
);
```

SUPPLIERS TABLE

```
CREATE TABLE Suppliers (
   supplier_id INT PRIMARY KEY,
   supplier_name VARCHAR(100),
   phone VARCHAR(15),
   email VARCHAR(100)
);
```

PURCHASES TABLE

```
CREATE TABLE Purchases (
   purchase_id INT PRIMARY KEY,
```

```
    supplier_id INT,
    item_code INT,
    purchase_date DATE,
    quantity INT,
    total_cost DECIMAL(10,2),
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(supplier_id),
    FOREIGN KEY (item_code) REFERENCES Inventory(item_code)
);
```

SAMPLE DATA

INSERT INTO Customers VALUES

(1,'Aarav Kumar','9876500001','aarav@gmail.com','Hyderabad'),

(2,'Priya Sharma','9876500002','priya@gmail.com','Secunderabad'),

(3,'Rahul Reddy','9876500003','rahul@gmail.com','Hyderabad'),

(4,'Sneha Patel','9876500004','sneha@gmail.com','Warangal'),

(5,'Vikram Rao','9876500005','vikram@gmail.com','Hyderabad');

INSERT INTO Menu_Items VALUES

(101,'Paneer Tikka','Starter',280.00,'Available'),

(102,'Chicken Biryani','Main Course',350.00,'Available'),

(103,'Veg Fried Rice','Main Course',240.00,'Available'),

(104,'Masala Dosa','Main Course',180.00,'Available'),

(105,'Gulab Jamun','Dessert',120.00,'Out of Stock');

INSERT INTO Orders VALUES

(1001,1,'2025-10-02',630.00,'Completed'),

(1002,2,'2025-10-05',350.00,'Placed'),

(1003,1,'2025-10-11',460.00,'Completed'),

(1004,3,'2025-10-18',700.00,'Completed'),

(1005,4,'2025-10-25',120.00,'Cancelled');

INSERT INTO Order_Items VALUES

(5001,1001,101,1,280.00),

(5002,1001,102,1,350.00),

(5003,1002,102,1,350.00),

(5004,1003,103,1,240.00),

(5005,1003,104,1,180.00),

(5006,1004,102,2,700.00);

INSERT INTO Inventory VALUES

(201,'Rice',40,'kg',10),

(202,'Chicken',25,'kg',8),

(203,'Paneer',12,'kg',5),

(204,'Cooking Oil',18,'liters',6),

(205,'Sugar',4,'kg',5);

INSERT INTO Suppliers VALUES

(301,'Fresh Foods Suppliers','9000000001','freshfoods@gmail.com'),

(302,'Hyderabad Agro','9000000002','agro@gmail.com'),

(303,'Daily Dairy Supply','9000000003','dairy@gmail.com'),

(304,'Kitchen Essentials','9000000004','kitchen@gmail.com'),

(305,'City Wholesale','9000000005','citywholesale@gmail.com');

INSERT INTO Purchases VALUES

(4001,301,201,'2025-10-01',50,3000.00),

(4002,302,202,'2025-10-03',20,7000.00),

(4003,303,203,'2025-10-04',10,2500.00),

(4004,304,204,'2025-10-08',25,3500.00),

(4005,305,205,'2025-09-20',10,600.00);

BASIC QUERIES

Display all menu items with price greater than 300.

```
SELECT * FROM Menu_Items
WHERE price > 300;
```

List all customers who placed orders in October 2025.

```
SELECT DISTINCT c.*
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2025-10-01'
AND o.order_date < '2025-11-01';
```

Find all menu items marked as “Out of Stock.”

```
SELECT * FROM Menu_Items
WHERE availability_status = 'Out of Stock';
```

Show details of all completed orders.

```
SELECT * FROM Orders
WHERE status = 'Completed';
```

JOIN QUERIES

List customer names with their ordered item names.

```
SELECT c.name, m.item_name
FROM Customers c
JOIN Orders o ON c.customer_id = o.customer_id
```

```
JOIN Order_Items oi ON o.order_id = oi.order_id
JOIN Menu_Items m ON oi.item_id = m.item_id;
```

Display order details with item name and quantity.

```
SELECT o.order_id, m.item_name, oi.quantity, oi.subtotal
FROM Orders o
JOIN Order_Items oi ON o.order_id = oi.order_id
JOIN Menu_Items m ON oi.item_id = m.item_id;
```

Show all orders with total amount and customer details.

```
SELECT o.order_id, o.order_date, o.total_amount,
    c.customer_id, c.name, c.phone
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id;
```

List suppliers with ingredients they supply.

```
SELECT s.supplier_name, i.ingredient_name
FROM Suppliers s
JOIN Purchases p ON s.supplier_id = p.supplier_id
JOIN Inventory i ON p.item_code = i.item_code;
```

Get details of menu items used in orders along with their category.

```
SELECT DISTINCT m.item_id, m.item_name, m.category
FROM Menu_Items m
JOIN Order_Items oi ON m.item_id = oi.item_id;
```

AGGREGATE QUERIES

Find total revenue earned.

```
SELECT COUNT(*) AS total_orders
FROM Orders
```

```
WHERE order_date >= '2025-10-01'
  AND order_date < '2025-11-01';
```

Count total number of orders placed in a month.

```
SELECT SUM(total_amount) AS total_revenue
FROM Orders
WHERE status = 'Completed';
```

Get the average order amount.

```
SELECT AVG(total_amount) AS average_order_amount
FROM Orders;
```

Show the most expensive dish on the menu.

```
SELECT *FROM Menu_Items
ORDER BY price DESC
LIMIT 1;
```

Find the total quantity of each ingredient in stock

```
SELECT ingredient_name, quantity_in_stock, unit
FROM Inventory;
```

SUBQUERIES

Find customers who haven’t placed any orders.

```
SELECT *
FROM Customers
WHERE customer_id NOT IN (
   SELECT customer_id FROM Orders );
```

Retrieve menu items that were never ordered.

```
SELECT *
FROM Menu_Items
WHERE item_id NOT IN (
   SELECT item_id FROM Order_Items
);
```

Get ingredients below reorder level.

```
SELECT *
FROM Inventory
WHERE quantity_in_stock < reorder_level;
```

Find suppliers who haven’t made any recent deliveries.

```
SELECT *
FROM Suppliers
WHERE supplier_id NOT IN (
   SELECT DISTINCT supplier_id
   FROM Purchases
   WHERE purchase_date >= '2025-10-01'
);
```

Get orders whose total amount is above average.

```
SELECT *
FROM Orders
WHERE total_amount > (
   SELECT AVG(total_amount)
   FROM Orders
);
```

UPDATE AND DELETE QUERIES

Update menu item availability after an order is completed.

```
UPDATE Menu_Items
SET availability_status = 'Out of Stock'
WHERE item_id = 105;
```

Change supplier contact details.

```
UPDATE Suppliers
SET phone = '9111111111',
   email = 'newcontact@gmail.com'
WHERE supplier_id = 301;
```

Update ingredient stock after new purchase.

```
UPDATE Inventory
SET quantity_in_stock = quantity_in_stock + 20
WHERE item_code = 202;
```

Delete cancelled orders.

```
DELETE FROM Orders
WHERE status = 'Cancelled';
```

Delete customers with no order history

```
DELETE FROM Customers
WHERE customer_id NOT IN (
   SELECT customer_id FROM Orders
);
```

ADVANCED (TRIGGERS, VIEWS, PROCEDURES)

TRIGGER:

Reduce inventory quantity automatically when an order is placed.

```
DELIMITER //

CREATE TRIGGER reduce_inventory_after_order_item
AFTER INSERT ON Order_Items
FOR EACH ROW
BEGIN
   UPDATE Inventory
   SET quantity_in_stock = quantity_in_stock - NEW.quantity
   WHERE item_code = NEW.item_id;
END //

DELIMITER ;
```

VIEW:

Show combined order report (Customer, Items, Total Amount).

```
CREATE VIEW combined_order_report AS
SELECT o.order_id, o.order_date, c.customer_id,

c.name AS customer_name,

GROUP_CONCAT(m.item_name ORDER BY m.item_name SEPARATOR ', ')
AS items, o.total_amount, o.status

FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id
JOIN Order_Items oi ON o.order_id = oi.order_id
JOIN Menu_Items m ON oi.item_id = m.item_id
GROUP BY o.order_id, o.order_date, c.customer_id, c.name,
o.total_amount, o.status;

SELECT * FROM combined_order_report;
```

PROCEDURE:

Generate daily sales summary.

```
DELIMITER //

CREATE PROCEDURE DailySalesSummary(IN p_date DATE)
BEGIN
  SELECT
    p_date AS sales_date,
    COUNT(*) AS completed_orders,
    COALESCE(SUM(total_amount), 0) AS total_sales,
    COALESCE(AVG(total_amount), 0) AS average_order_amount
  FROM Orders
  WHERE order_date = p_date
   AND status = 'Completed';
```

```
END //

DELIMITER ;

CALL DailySalesSummary('2025-10-11');
```

TRANSACTION:

Add order and order items only if all items are available.

```
START TRANSACTION;

INSERT INTO Orders
(order_id, customer_id, order_date, total_amount, status)
VALUES
(1010, 5, '2025-10-30', 700.00, 'Placed');

-- Add order items only after availability checks
INSERT INTO Order_Items
(order_item_id, order_id, item_id, quantity, subtotal)
SELECT 5010, 1010, 102, 2, 700.00
FROM Menu_Items
WHERE item_id = 102
  AND availability_status = 'Available';

-- If all required items were inserted successfully:
COMMIT;

-- Otherwise:

ROLLBACK;
```